LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT)

Modul 7

Oleh :

Nama : Rena Anggraini

NPM : 25782022

Kelas : TRI (3A)

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI

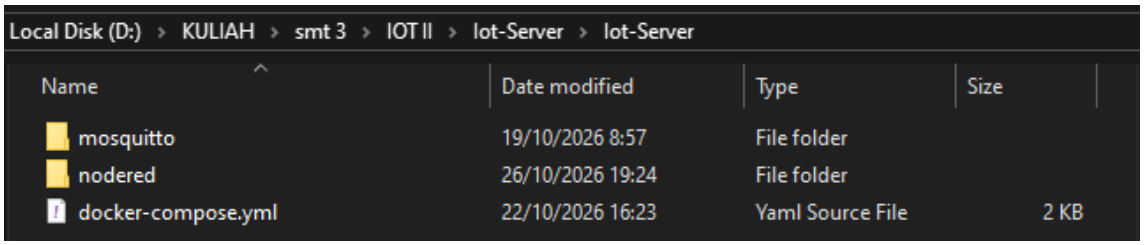
TEKNOLOGI REKAYASA INTERNET

POLITEKNIK NEGERI LAMPUNG

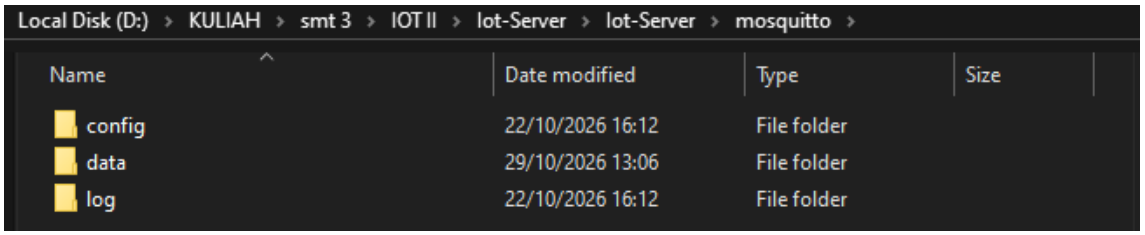
TAHUN 2026

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

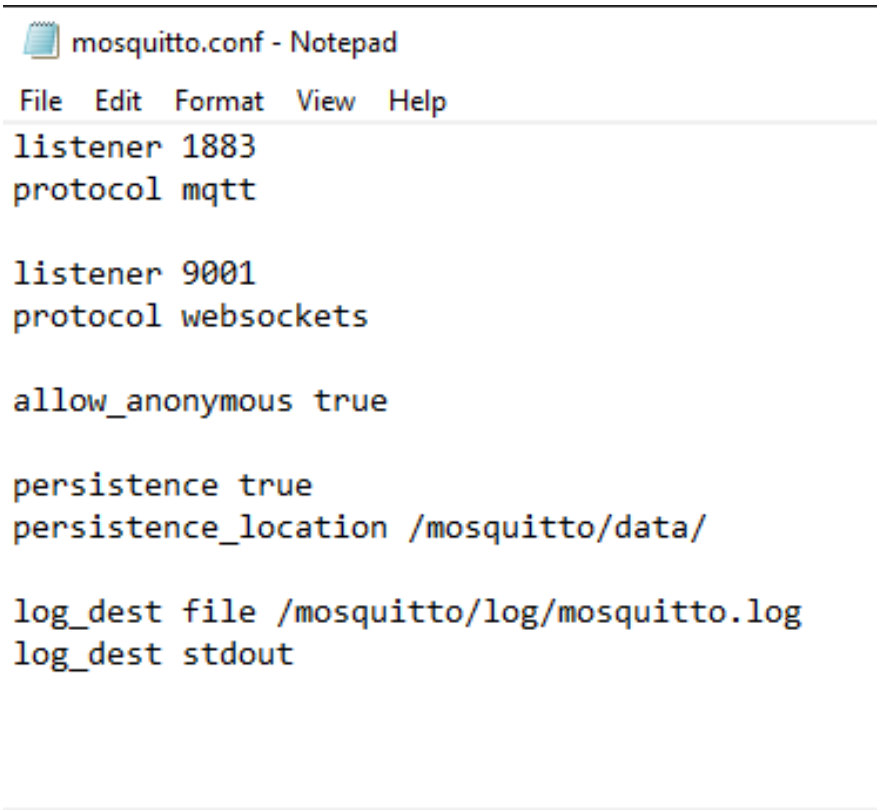
Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: iot-server (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).



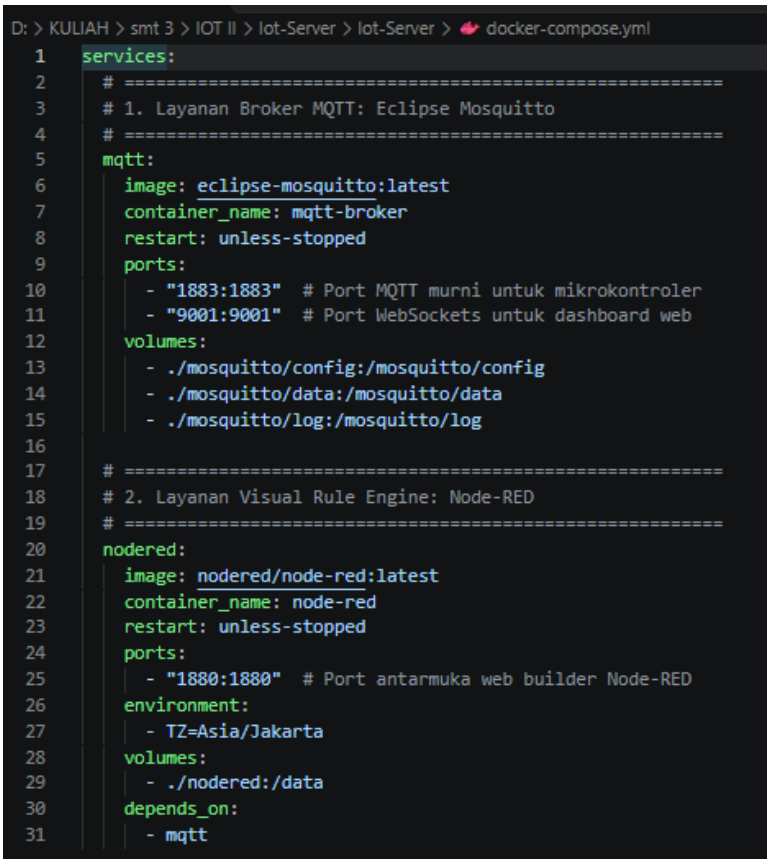
Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:



Buka teks editor (VS Code atau Notepad), buat berkas baru bernama mosquitto.conf, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:



Kembali ke folder induk iot-server, buat berkas baru bernama docker-compose.yml, lalu salin konfigurasi layanan terpadu berikut:



Jalankan Terminal (Linux/macOS) atau PowerShell / Command Prompt (Windows), arahkan direktori aktif ke dalam folder iot-server:

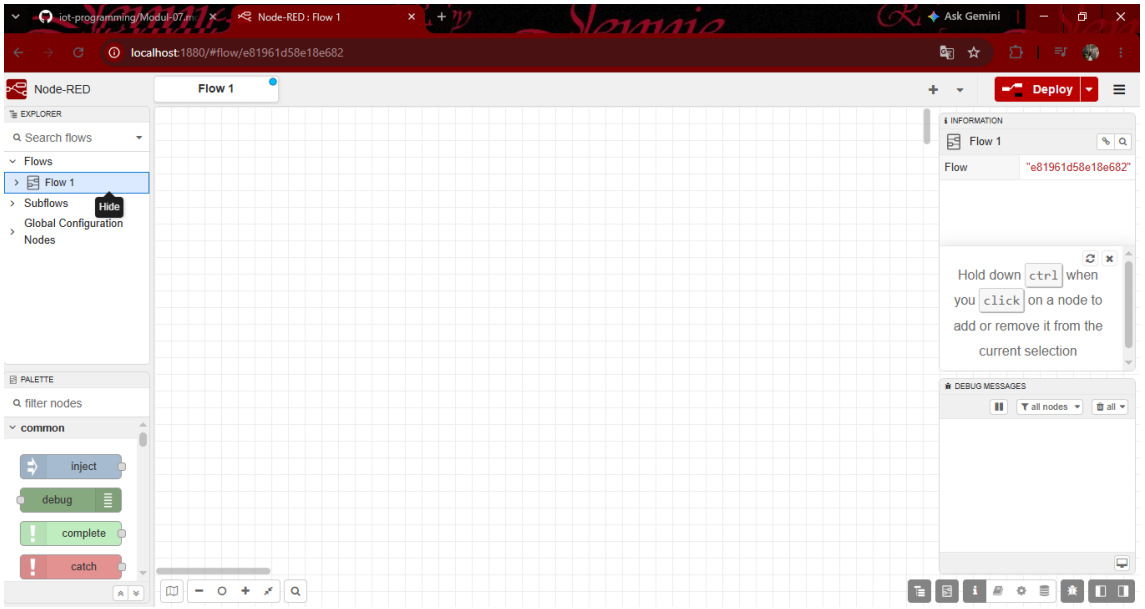
Eksekusi perintah berikut untuk mengunduh (pull) image dan menjalankan kontainer di latar belakang (detached mode):

```
D:\KULIAH\smt 3\IOT II\Iot-Server\Iot-Server>docker compose up -d
[+] up 2/2
 ⠿ Container mqtt-broker Running
 ⠿ Container node-red    Running
```

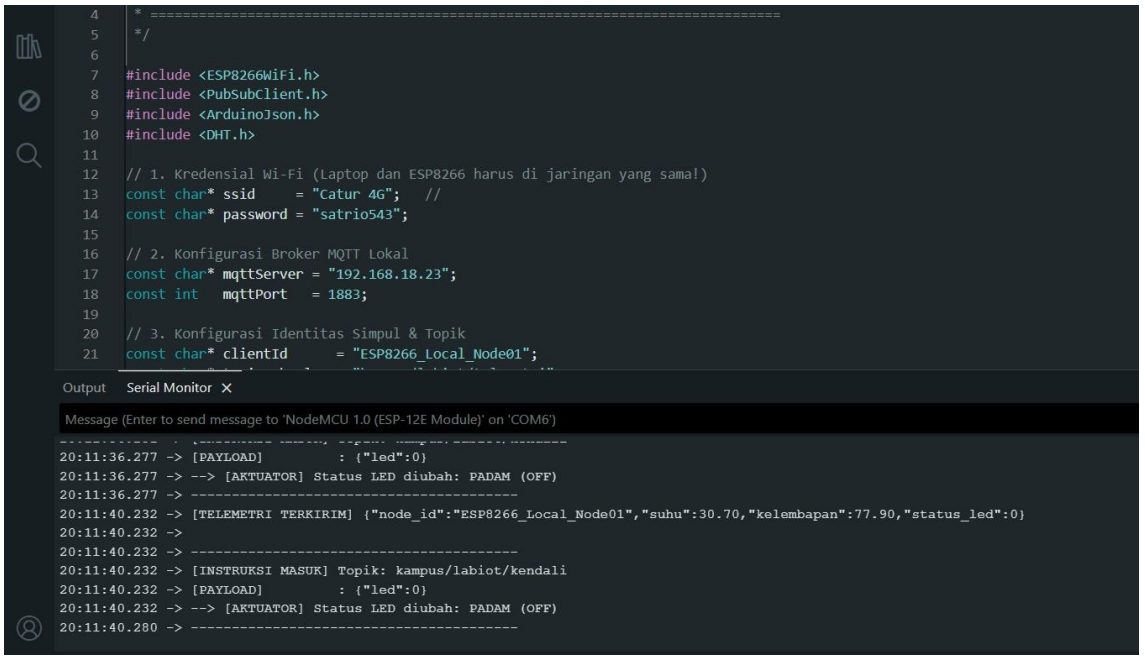
Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

```
D:\KULIAH\smt 3\IOT II\Iot-Server\Iot-Server>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED      STATUS                    PORTS
                                                                                NAMES
83e3ddbfb264   nodered/node-red:latest   "./entrypoint.sh"        9 days ago   Up 18 minutes (healthy)   0.0.0.0:1880->
1880/tcp, [::]:1880->1880/tcp                                                   node-red
a88a7b6c589f   eclipse-mosquitto:latest  "/docker-entrypoint.…"   9 days ago   Up 18 minutes             0.0.0.0:1883->
1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker
```

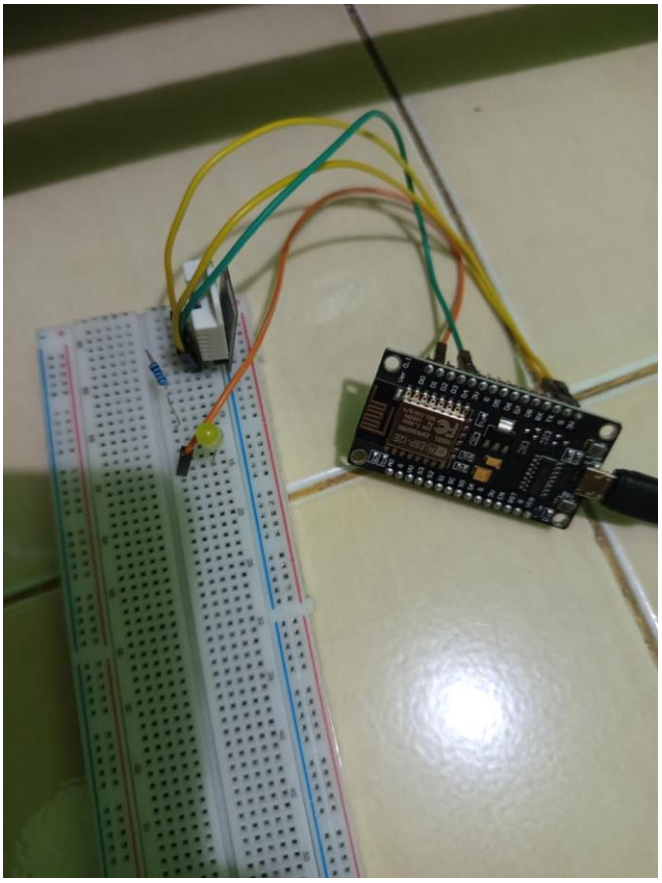
Buka peramban web pada laptop Anda dan ketikkan alamat URL: http://localhost:1880. Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!



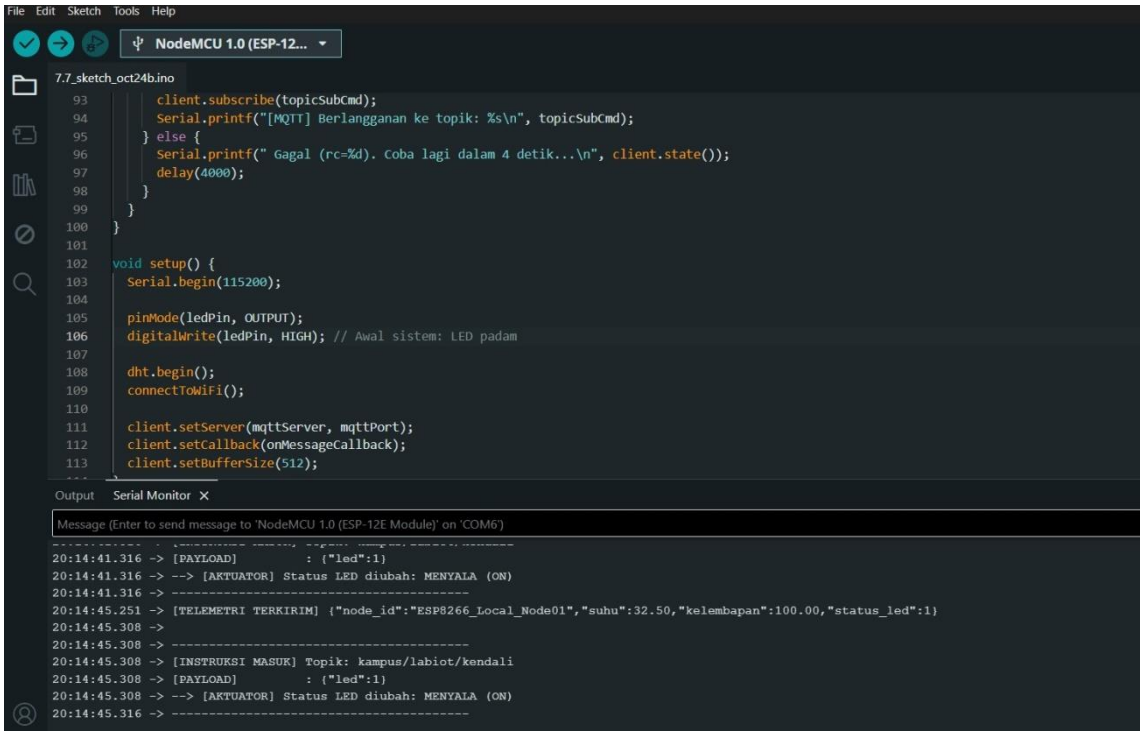
7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal



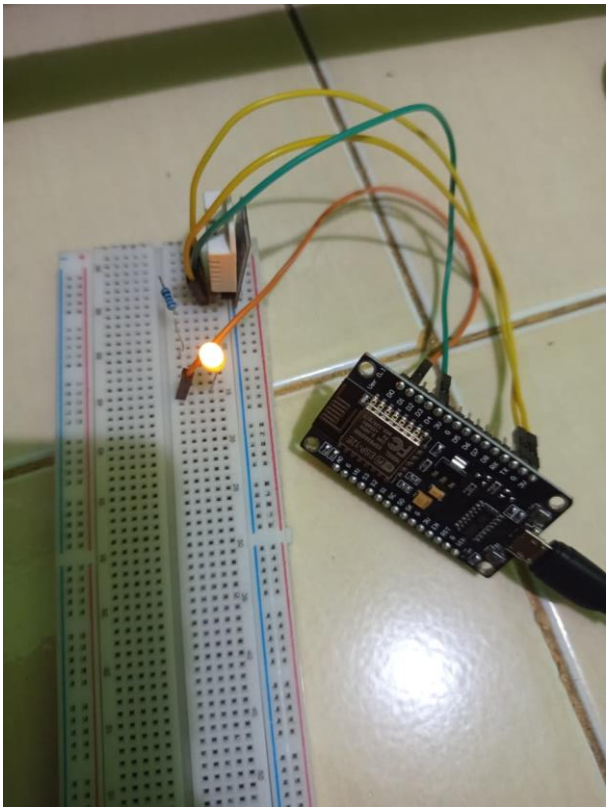
Serial Monitor Led Off



Perangkat Led Off

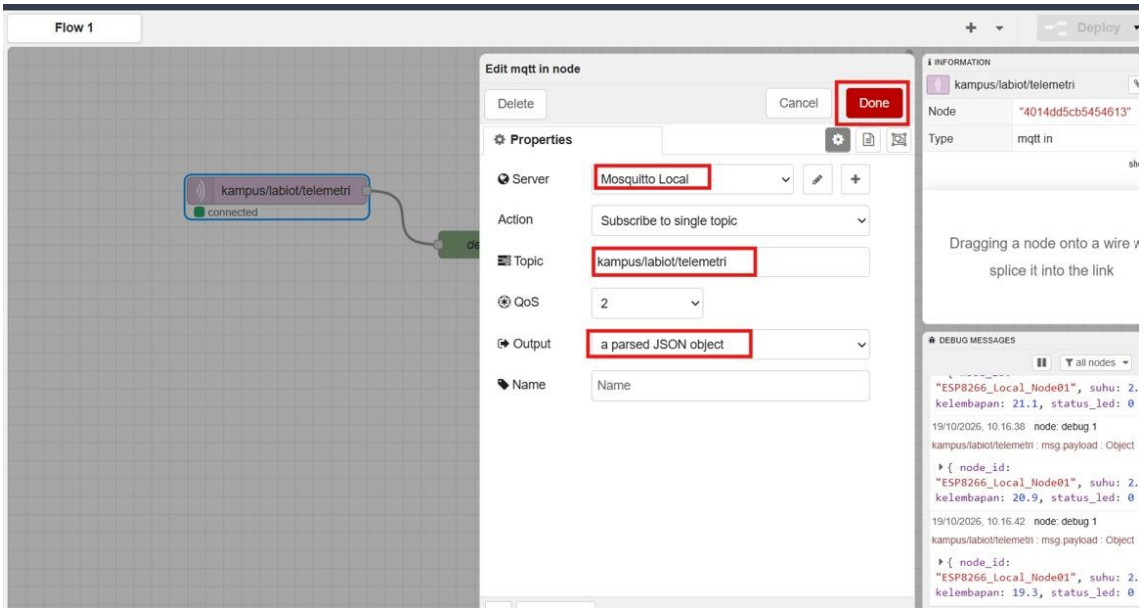


Serial Monitor Led On

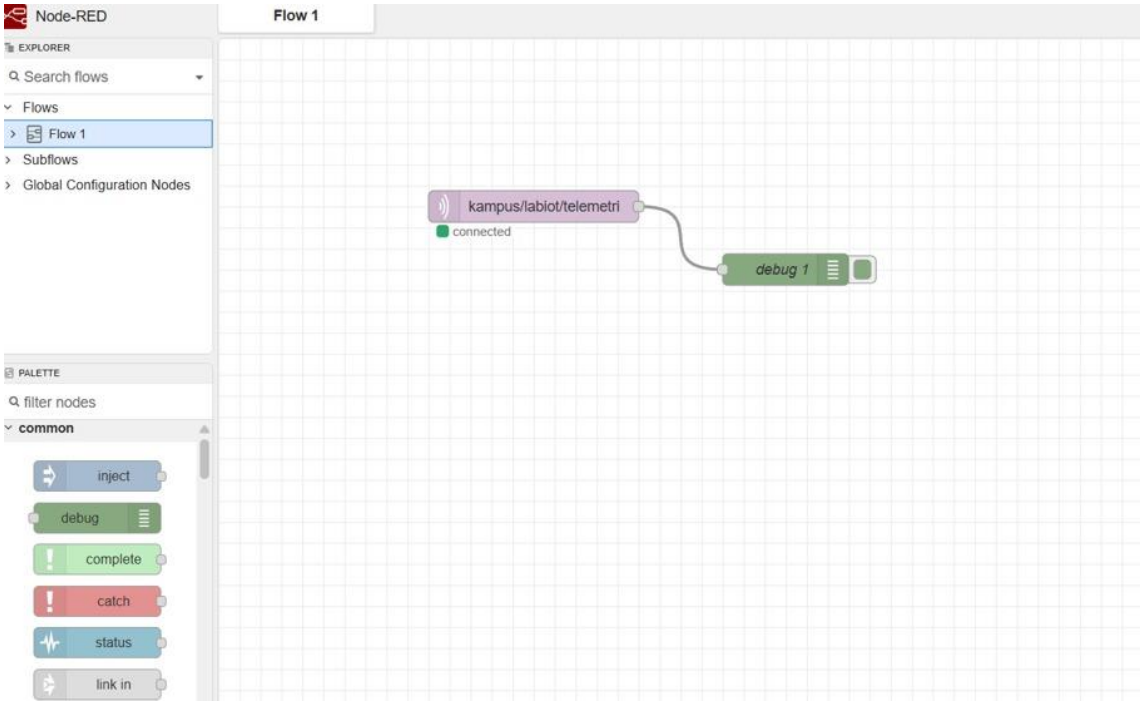


Perangkat Led On

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

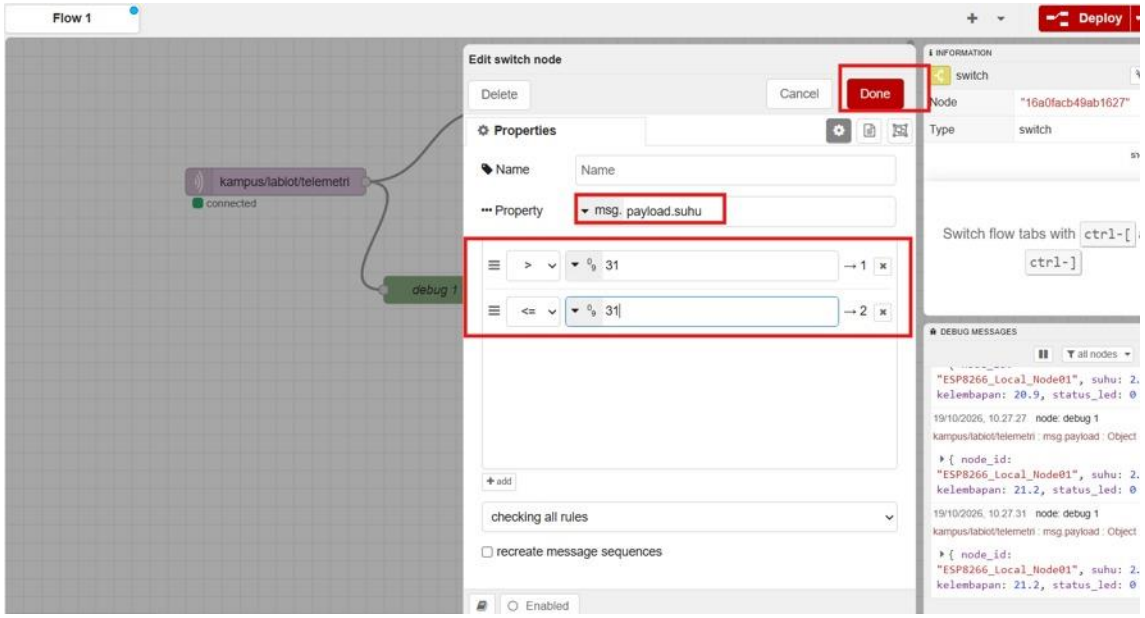


Mengatur Setting Mqtt

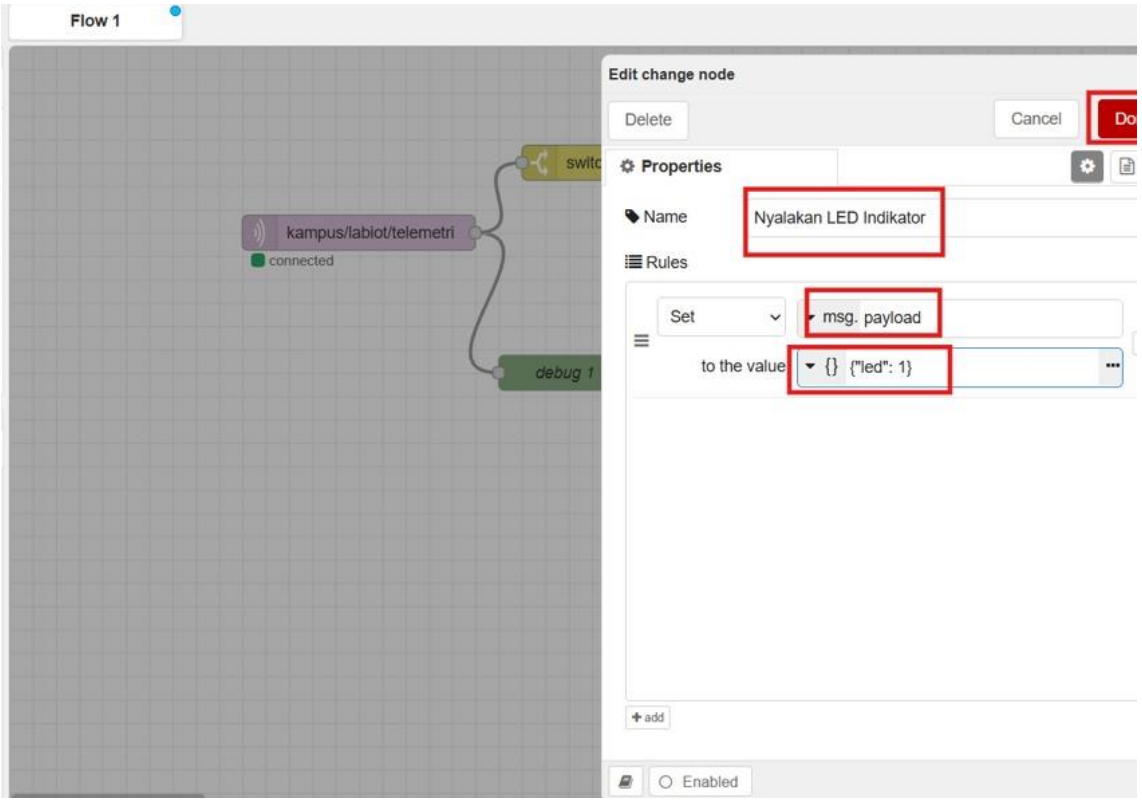


Hasil Node Red

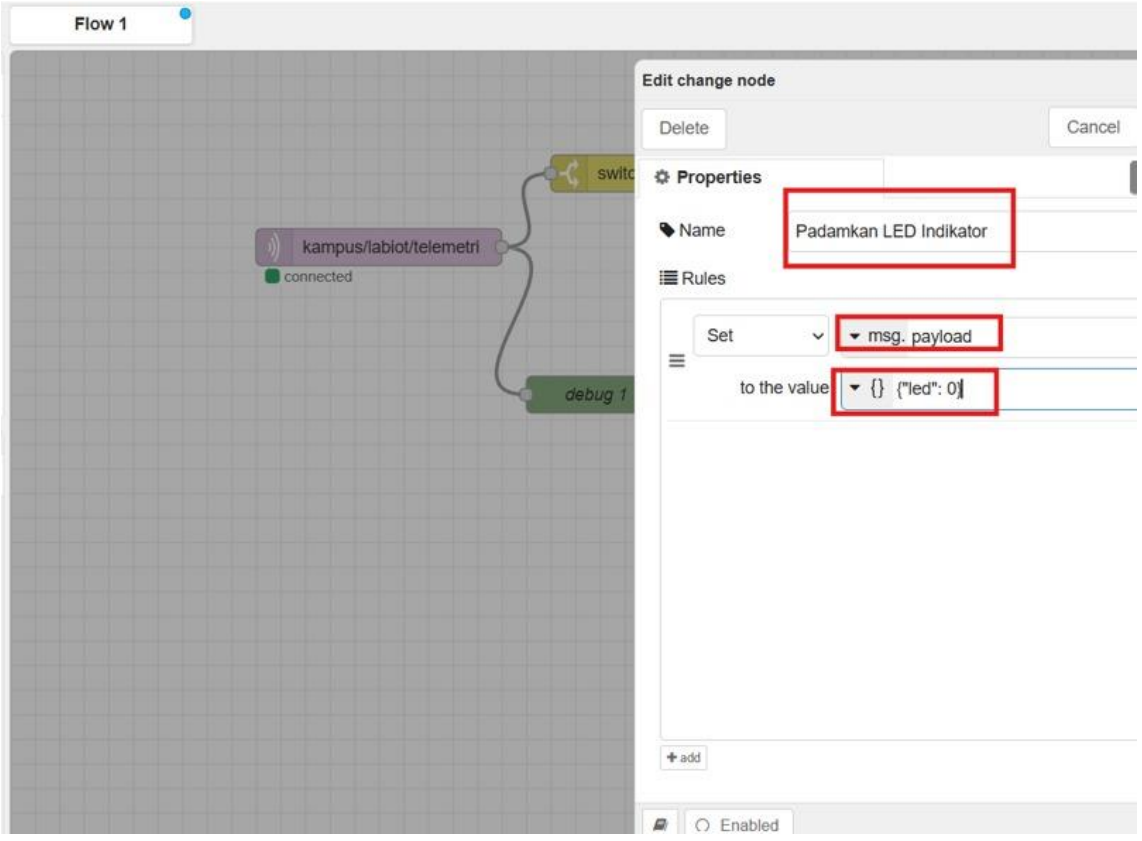
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup



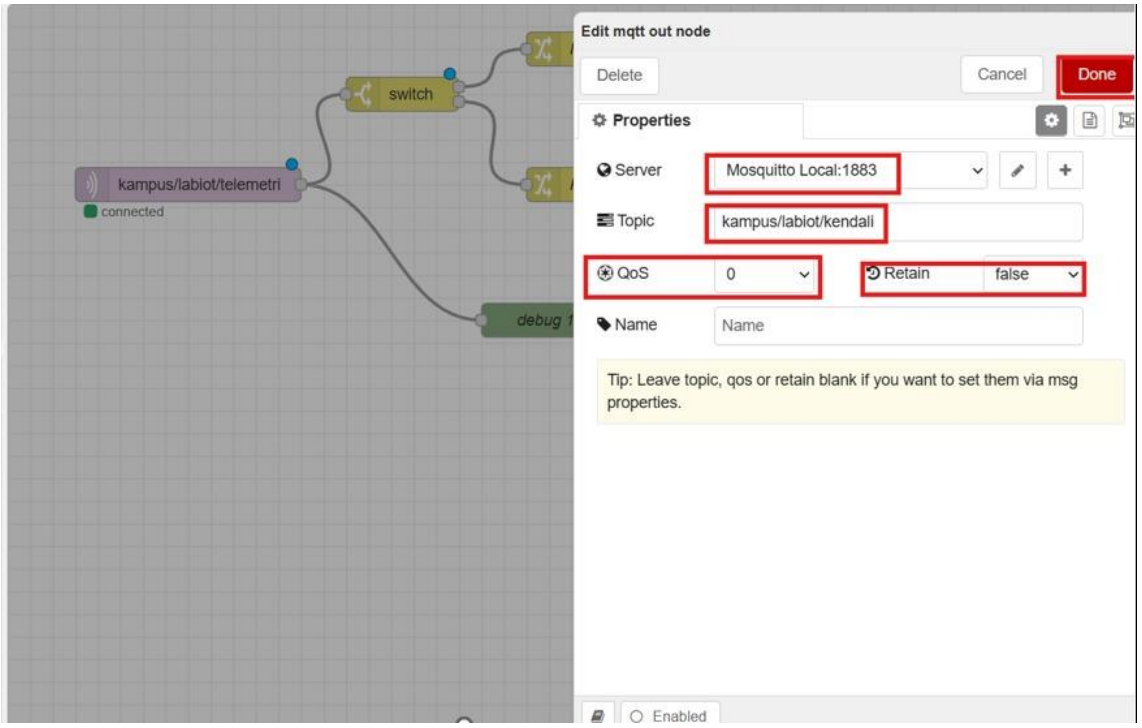
Settiing Switch



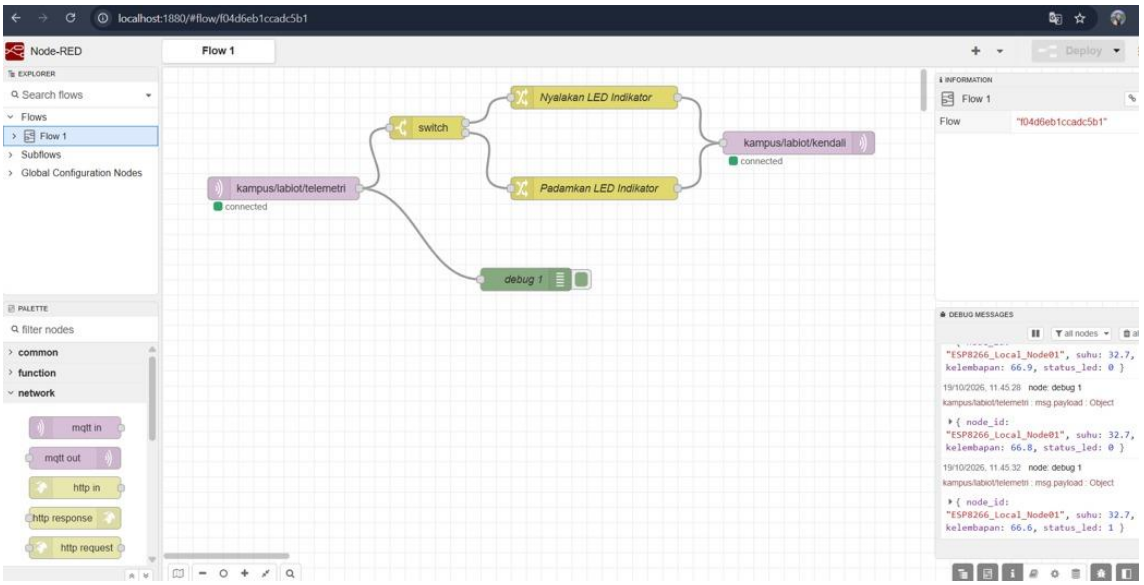
Setting Change untuk menghidupkan led



Setting Change untuk mematiakan Led



Setting Mqtt Out



Hasil akhir, jika sushu lebih dari 31 Led akan menyala

7.6 Latihan

- Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):

  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (Connection Refused)?

  Jawab:

  Setiap Kontainer Docker berjlaan didalam network namespace-nya sendiri yang terisolasi. Ketika menulisakan localhost di dalam kontainer Node-RED, sistem mengartikannya sebagai "perangkat lokal kontainer Node-RED itu sendiri", bukan laptop host Anda ataupun kontainer Mosquitto. Karena broker MQTT berjalan di kontainer terpisah, port 1883 tidak aktif di dalam namespace Node-RED, sehingga koneksi ditolak (Connection Refused).

  ii. Jelaskan bagaimana mekanisme Docker Embedded DNS Server dan Bridge Network bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jawab:

Ketika kontainer-kontainer berada di dalam jaringan custom bridge yang sama, Docker secara otomatis menjalankan Embedded DNS Server internal Saat Node-RED mencoba menghubungi host bernama mqtt, Docker DNS akan secara otomatis melakukan resolve nama tersebut menjadi alamat IP internal kontainer Mosquitto di jaringan bridge yang sama, memungkinkan komunikasi langsung tanpa perlu mengetahui alamat IP statisnya.

- Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):

  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

Jawab:

Saat Node-RED mencoba menghubungi host bernama mqtt, Docker DNS akan secara otomatis melakukan resolve nama tersebut menjadi alamat IP internal kontainer Mosquitto di jaringan bridge yang sama, memungkinkan komunikasi langsung tanpa perlu mengetahui alamat IP statisnya.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain broadcast / subnet yang sama?

Jawab:

- Hubungkan ESP8266 ke jaringan Wi-Fi yang sama persis dengan yang digunakan laptop (misalnya Wi-Fi router rumah dengan subnet 192.168.1.x).
- Hubungkan laptop Anda ke hotspot HP yang sama dengan ESP8266 agar kedua perangkat mendapatkan IP di dalam subnet yang selaras (misalnya sama-sama di 192.168.43.x).
- Pastikan pengaturan firewall laptop (seperti UFW di Linux atau Windows Defender Firewall) mengizinkan lalu lintas masuk pada port MQTT (1883).

- Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):

  Pada sistem kendali dua posisi (Bang-Bang), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C(misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (flickering / hunting) yang mengganggu stabilitas visual sistem.

  Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:

  - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).Tuliskan kode JavaScript yang Anda masukkan ke dalam Function Node tersebut!

Jawab:

Untuk mencegah flickering (nyala-padam cepat) di sekitar ambang batas 31.0°C, kita menggunakan variabel state untuk mengingat kondisi terakhir lampu (context.get / context.set). Lampu hanya akan menyala jika suhu naik melewati 31.5°C dan baru akan padam jika suhu turun di bawah 30.0°C.

```
// Mengambil nilai suhu dari msg.payload (asumsi angka)
let temp = msg.payload;

// Mengambil status LED sebelumnya dari konteks (default 0 jika belum ada)
let ledState = context.get("ledState") || 0;

// Logika Histeresis
if (temp > 31.5) {
    ledState = 1; // Nyalakan LED jika melebihi batas atas
} else if (temp < 30.0) {
    ledState = 0; // Padamkan LED jika turun di bawah batas bawah
}
// Jika suhu berada di antara 30.0 dan 31.5, ledState dipertahankan (tidak berubah)

// Simpan kembali status LED ke konteks
context.set("ledState", ledState);

// Format output payload sesuai kebutuhan sistem
msg.payload = { "led": ledState };

return msg;
```

- Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.
  - Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
  - Jelaskan perbedaan mendasar antara Bind Mount dan Named Volume pada Docker!

Jawab:

Seluruh data di dalam kontainer Docker bersifat ephemeral (sementara). Tanpa adanya volume mount (./nodered:/data), file konfigurasi dan kanvas flows Node-RED tersimpan di dalam container layer yang akan hilang sepenuhnya saat perintah docker compose down dijalankan. Ketika kontainer dinyalakan kembali, Node-RED akan kembali ke kondisi awal yang kosong seperti instalasi baru.

7.7 TUGAS

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
D:\KULIAH\smt 3\IOT II\Iot-Server>
D:\KULIAH\smt 3\IOT II\Iot-Server>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

D:\KULIAH\smt 3\IOT II\Iot-Server>
```

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

mosquitto.conf - Notepad

File Edit Format View Help

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous false
password_file /mosquitto/config/password.txt

persistence true
persistence_location /mosquitto/data/

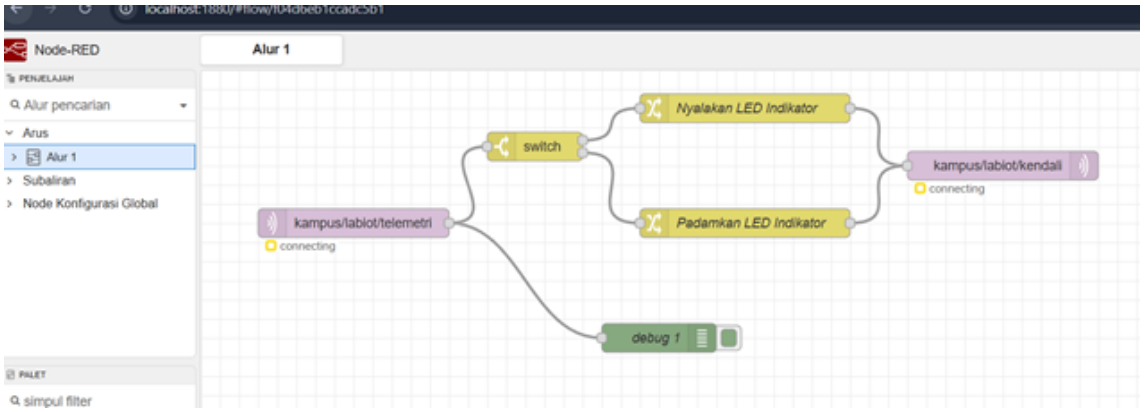
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

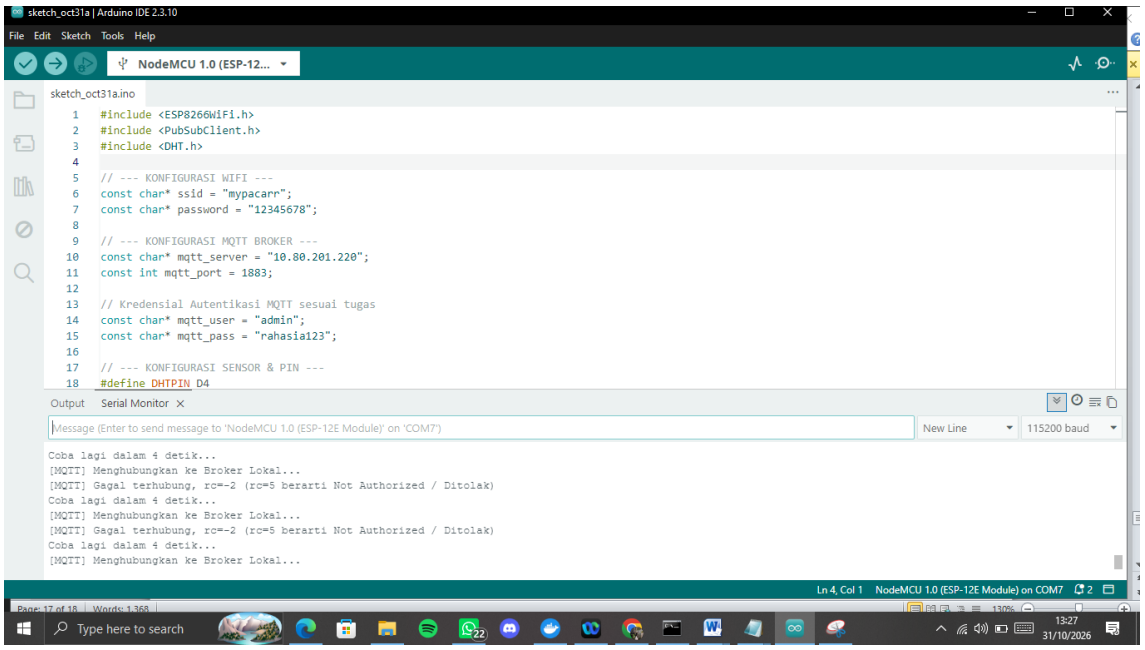
3. Memuat Ulang Layanan Broker:

```
D:\KULIAH\smt 3\IOT II\Iot-Server>docker restart mqtt-broker
mqtt-broker

D:\KULIAH\smt 3\IOT II\Iot-Server>
```

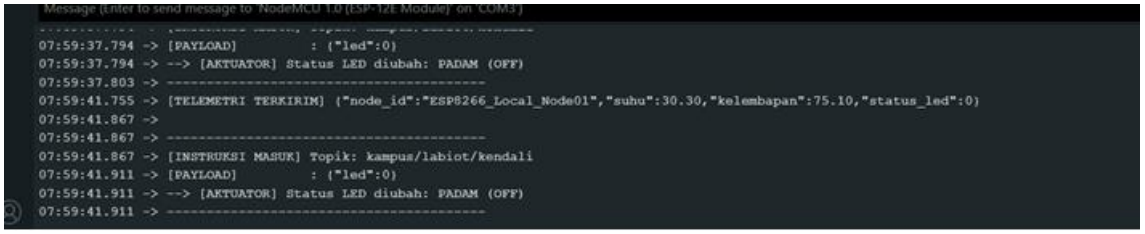
4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):





5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

- Pemutakhiran Skrip Program NodeMCU ESP8266



- Pemutakhiran Konfigurasi Kredensial pada Node-RED

